

Agent = Harness(Model): Five Mechanistic Primitives of LLM Agent Harnesses

Chase Dovey
Average Joes Lab

October 2026 Preprint

Abstract

The software that surrounds a large language model (LLM) in an agent is described in the literature and in practice under many names: framework, scaffold, runtime, orchestration layer, harness. Recent work shows that this layer accounts for a substantial share of agent performance and that comparisons which omit it are misleading. It has not, however, been characterized in terms of its constituent parts. This paper provides such a characterization.

We separate an agent into two components. The **model** is treated as a fixed function, $TokensOut = Model(TokensIn)$. The **harness** comprises everything else, so that **Agent = Harness(Model)**. We argue that every harness is composed of **five mechanistic primitives**:

- **control flow**, which sequences the other four and determines termination;
- **input**, which gathers what is sent to the model, from a person or from the environment;
- **context**, which determines what the request contains and fits it to the model’s context window;
- **the model interface**, which sends the request and receives the response;
- **output**, which handles the response, either by presenting it to a person or by executing it as a tool call.

Each primitive is defined both positively, by what it does, and negatively, by what it never does, so that the five are mutually exclusive.

The central argument is constructive. Starting from a single API call and adding one primitive at a time, we show that the deficiency at each step is precisely the next primitive. The result is a working agent of 91 lines of Python and a 148-line system prompt, containing nothing outside the five.

Two results follow.

1. **Production concerns reduce to the primitives.** Sandboxing, guardrails, observability, resilience, performance and evaluation are shown to be hardening within the primitives they touch, not additional primitives.
2. **The decomposition holds on production systems.** We decompose three production coding agents (Claude Code, OpenAI Codex CLI and opencode) and find no component that requires a sixth primitive. The study also identifies where the original boundary rules were underspecified, and we refine them accordingly.

A case study, in which the constructed agent maintained its own teaching material and reviewed this paper across ten sessions, drawing on its own memory, illustrates the use of the decomposition for failure diagnosis.

Companion artifact (an open course with runnable code): <https://github.com/averagejoeslab/building-agents> [1].

1 Introduction

Descriptions of LLM agents use inconsistent terminology for the software around the model: *framework*, *scaffold*, *runtime*, *orchestration layer* and *harness* are all in common use. When the underlying systems are examined, however, they share a common mechanism:

1. a model is called;
2. some component determines what the call contains;
3. some component acts on what the call returns;
4. some component decides whether to call again.

The terminology varies; the mechanism does not.

The absence of a shared decomposition has practical costs.

- **Diagnosis.** Without names for an agent’s parts, a failure cannot be attributed to a part.
- **Comparison.** Systems described in different terms cannot be compared directly.
- **The product landscape.** The same few ideas recur as separate products: memory layers, model gateways, tool protocols, tracing platforms, sandboxes and evaluation suites. Without a map, it is unclear which of them are new and which are existing ideas under new names.

Recent work establishes that this layer matters. Zhang et al. [2] argue that, among comparable frontier models, harness configuration can account for more of the variance in performance than the choice of model, and that agents should therefore not be compared without disclosing their harnesses. Fan et al. [3] hold a coding harness’s loop fixed and vary three of its parts (planning, the action space and context management), finding that each alters agent behavior. Both treat the harness as a primary determinant of agent behavior. Fan et al. select parts of a harness to vary; the present work asks what the complete set of parts is.

Our approach is **reductive**: we decompose an agent until its parts are no longer shared across systems. It is also **mechanistic**: each part is defined by its function on the path from request to response, not by its name or by the product that provides it.

Contributions. Contributions 1–3 constitute the core of the paper; contributions 4–6 follow from them.

1. **A two-component view of agents**, Agent = Harness(Model), in which the model is a fixed function and all else is harness (§2).
2. **Five primitives of a harness**, each with a positive and a negative definition, together with rules for resolving cases at their boundaries (§3–4).
3. **A constructive argument for sufficiency**: a working agent built one primitive at a time, in which each step’s deficiency is the next primitive (§5).
4. **A reduction of production concerns** to hardening within the primitives (§6).
5. **A decomposition of three production harnesses**, Claude Code, Codex and opencode, which finds no need for a sixth primitive and refines the boundary rules (§7).
6. **A case study** in which the constructed agent, working across sessions from its own memory, maintained its teaching material and reviewed this paper (§8).

This is a conceptual and analytical contribution, in the tradition of frameworks such as CoALA [4]: its evidence is a construction, a decomposition of existing systems, and a case study, and it reports no quantitative experiments. Measuring how much each primitive contributes to differences in performance is left to future work (§9.5).

The paper is accompanied by a companion artifact, *harness-engineering* [1], an open course with runnable code. The artifact builds the harness of §5 and the production layers of §6 incrementally and retains full transcripts of every run. The paper is self-contained: every listing and run on which the argument depends is reproduced in Appendices A–D.

2 Two components: the model and the harness

2.1 The model

The model predicts. Given a sequence of tokens, it produces a distribution over the next token:

$$\text{TokensOut} = \text{Model}(\text{TokensIn})$$

Viewed from outside, the model is a function. It retains no state between calls, cannot read from or act on its environment, and cannot initiate a further call. Everything it knows about the present moment is contained in its input. The model is the product of *model development* (training data, compute, and methods for training and evaluation), a separate discipline whose output we treat as fixed.

2.2 The harness

The harness comprises everything else. It determines how inputs are gathered, how they are presented to the model, how the model is called, how its outputs are handled, and how information flows among these steps:

$$\text{Agent} = \text{Harness}(\text{Model})$$

Harness engineering is the construction of this second component.

The division is deliberately coarse. Anything that is not the model’s weights is harness: a system prompt, a tool, a retry policy, a memory store. Two agents built on the same model differ only in their harnesses; a comparison that omits the harness therefore omits the variable under study.

2.3 Method

The decomposition proceeds by repeated application of a single question:

What is it, and by what mechanistic primitives does it work?

Applied to an agent, the question yields a model and a harness. Applied to a harness, it yields five primitives (§3). Applied once more, to a primitive, it yields answers that are no longer shared across systems: one harness’s input reads a terminal, another’s a chat channel, a third’s a webhook. We take this as the stopping criterion. A primitive is the deepest level at which every harness still has the same parts.

Each primitive is given two definitions:

- a **positive** definition, stating what it does;
- a **negative** definition, stating what it never does and naming the primitive responsible for that function instead.

The negative definitions carry most of the analytical weight. They make the primitives mutually exclusive, so that each piece of harness code has exactly one home. A piece is assigned by **its function, not by its location in the code** or by its name (§4).

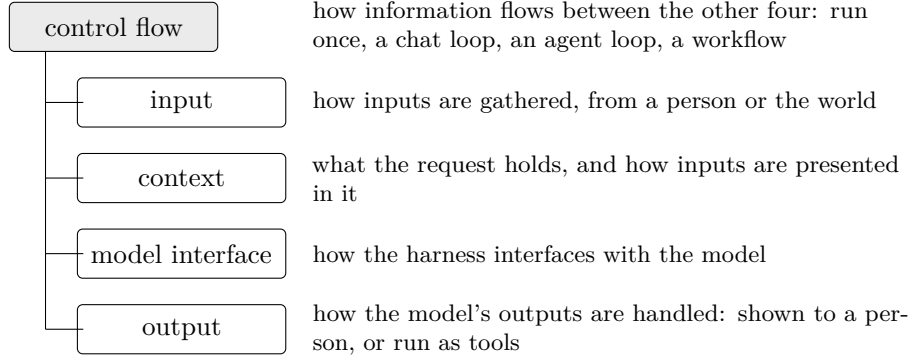


Figure 1: The five primitives of a harness. Control flow wraps the other four.

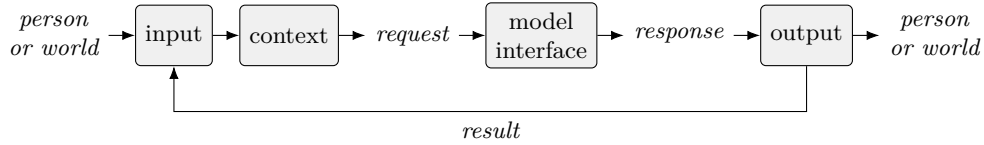


Figure 2: One pass through a harness. A tool's result comes back as input; control flow decides whether to go around again.

2.4 Scope

The primitives describe what happens on the path of a request at run time. Some software that ships with a harness never acts on that path, and we exclude it, as we exclude model development:

- **transport and hosting:** client/server splits, remote workspaces, the process serving a user interface;
- **persistence:** databases;
- **event wiring:** internal event buses;
- **configuration and packaging:** settings loaders, plugin installers.

These components carry, store or configure the primitives without constituting a step in a pass. Where such a component does act on the path, for example a saved transcript that the harness replays, that action is assigned like any other.

3 The five primitives

One pass through a harness follows a single path:

Input gathers what is to be sent. Context assembles it into a request and fits it to the model's window. The model interface sends the request and receives the response. Output handles the response, either presenting it to a person or executing a tool; a tool's result re-enters as input. Control flow determines what happens at the end of the path: another iteration, a turn for a person, or termination.

3.1 Model interface

Does: sends tokens to the model as a request and receives tokens as a response.

Never does: decide when to call (control flow), gather what is sent (input), determine how it is presented (context), or act on what is returned (output).

The model interface is the thinnest primitive, as a consequence of its position. The other four operate on the harness side; the model interface is the boundary itself, the parentheses in *Agent = Harness(Model)*. Its minimal form is a single SDK call. Richer implementations vary along several dimensions: the destination of the request (a hosted API, a local model, a gateway); its transport (an SDK, raw HTTP, a cross-provider translation layer); which model responds (a single model, or a backup when the first is unavailable); how the response arrives (whole or streamed); how many requests are sent (singly or in batches); failure handling (retries, backoff, timeouts); and request settings (output limits, reasoning effort, thinking summaries). All of these concern the delivery of one request and the receipt of one response.

3.2 Input

Does: gathers what is sent to the model, from a **person** (a task, a question, a message) or from **the environment** (the result of a tool execution: its output, its exit status, whether it failed).

Never does: determine how what it gathers is laid out in the request (context), send it (model interface), or decide whether to iterate (control flow).

Input is the harness's afferent pathway. Its sources include command-line arguments, prompts, pipes, chat applications, webhooks and schedules; its content may be text, images or files. Whatever the source, its function is to bring something from outside the harness to the edge of the request.

3.3 Output

Does: handles the model's response, either by **presenting** it to a person or by **executing** it as a tool call.

Never does: decide whether a tool's result is returned to the model (control flow), or determine what the next request contains (context).

Output is the efferent pathway. The model cannot act, but it can request an action: its response may contain a tool call, consisting of a name and arguments in a schema the harness has described to it. Output executes the call, and the result returns as input. The model requests; the harness acts. The design choices within output include which tools exist (one general tool or many specific ones); where they execute (locally, in a container, on a remote host, or at the provider); whether multiple calls execute sequentially or concurrently; when not to execute (a call truncated mid-generation is incomplete, and executing a partial command is worse than executing none); and how failures and hangs are handled.

Input and output are constructed independently but form two ends of a single exchange, which meets at the tool: output executes the call and input returns its result.

3.4 Control flow

Does: **sequences** the other four primitives, passing each one's product to the next, and **terminates**, deciding when to stop or return control to a person.

Never does: send the request (model interface), gather (input), present (context), or handle the response (output).

Control flow is the only primitive that contains the others. Its arrangement of them determines the kind of system that results.

Shape	Arrangement	Who determines the next step
Single call	input $\rightarrow$ call $\rightarrow$ output, then stop	no one; it is fixed
Chat loop	call, present, return to the person, repeat	the person
Workflow	the harness code fixes the sequence of calls (e.g., draft, evaluate, revise)	the harness code
Agent loop	call; while the model requests tools, execute them, return the results, call again	the model

The distinction between a *workflow* and an *agent* [5] is thus a property of control flow: which party determines the next step. Termination is an equal half of the definition. A loop without a defined termination condition has no bound on its cost. The minimal agent loop terminates when the model stops requesting tools; richer implementations also terminate on step limits, spending limits, refusals, or a person’s instruction.

3.5 Context

Does: before every call, **assembles** what the request contains and **fits** it to the model’s context window.

Never does: gather inputs (input), send the request (model interface), or decide when to call (control flow).

Each call begins without state. The model has no knowledge of its environment, the date, its own identity, its previous actions, or earlier instructions unless the harness includes them in the request. For this reason, a large share of what is built into harnesses is context. The following components are all means of determining what the model sees:

- **instructions:** the system prompt;
- **working memory:** the current session;
- **episodic memory:** prior sessions;
- **semantic memory:** durable facts;
- **procedural memory:** skills, recipes and playbooks;
- **retrieval:** search results inserted into the request;
- **compaction:** a summary substituted for history that no longer fits;
- **self-knowledge:** the agent’s own description or source.

Input and context are distinct. Input brings an item to the edge of the request; context determines whether it is included, where, in what form, and what is removed to accommodate it.

3.6 Summary

Primitive	Does	Never does
Model interface	sends the request, receives the response	decide when, gather, present, act
Input	gathers from a person or the environment	present, send, decide to iterate
Output	presents the response or executes a tool	decide whether results return, shape the next request
Control flow	sequences and terminates	send, gather, present, handle

Primitive	Does	Never does
Context	assembles and fits the request	gather, send, decide when

4 Boundary rules

A taxonomy is only as useful as its treatment of difficult cases. The following rules resolve the cases encountered in this work. Each follows from the negative definitions of §3. Several were made explicit by the decomposition study of §7; these are marked.

Assignment by function, not location. Code that determines what the model sees is context, even when it resides inside a tool. For example, `opcode` attaches nested `AGENTS.md` files to the `Read` tool’s result, and Claude Code’s `WebFetch` tool condenses a page with a separate model call before the model sees it. Conversely, an approval decision is control flow even when it executes within a tool’s runtime, as in `Codex`. (*Refined in §7.*)

The tool channel. In all three production harnesses studied, the model drives other primitives through tool calls: spawning a subagent, entering a planning mode, writing a task list, searching for tools, scheduling a wake-up, requesting a fresh context window. The tool call itself is output; its effect is assigned to the primitive it changes (control flow, context or input). (*From §7.*)

Backup versus routing. A change of model is classified by its trigger.

- If the change occurs because **the request could not be served** by the first model (it was unavailable, overloaded, or refused the request), it belongs to the **model interface**: it alters only how a single request obtains a response.
- If the change is **keyed to the work** (the task is simple; the current phase is planning), it is **routing**, a decision about the work, and belongs to **control flow**.

A fixed model for one class of request, such as compaction summaries, is a request setting and belongs to the model interface. Claude Code’s `opusplan` setting, which uses one model for planning and another for execution, is keyed to the phase and is therefore control flow. A content-classifier fallback that retries a flagged request on another model is model interface; retaining the second model for the remainder of the session is a separate, control-flow decision. (*Refined in §7.*)

Streaming. *Receiving* a response as a stream is the model interface; *rendering* each fragment as it arrives is output.

Parallelism. Executing several *tool calls* from one response concurrently is output, since it concerns how those calls are executed. Issuing several *model calls* concurrently, whether to divide a task or to vote, arranges the work and is control flow.

The reader of an artifact. A record kept **for the model**, such as a searchable episode log, is **context**. A record kept **for the operator**, such as a trace of every call, is **observability** (§6). An artifact produced **for the user**, such as a per-turn diff of changed files, is **output**. A single file may serve several readers; Claude Code’s session transcript is read by the operator and replayed by the harness on resumption. Each use is then assigned separately. (*Refined in §7.*)

The consumer of a person’s answer. When the **harness** consumes the answer, as with an approval prompt, the answer is a **control-flow** decision, not new input. When the **model** consumes it, as when the model poses a question and receives the answer as a tool result, it is **input** from a person. (*From §7: Codex’s `request_user_input`, opcode’s `question tool`, Claude Code’s `AskUserQuestion`.*)

Direct actions by a person. A shell command executed directly by the person (the `!cmd` syntax in Codex, Claude Code and opencode) brings environment state into the conversation and is input, although it reuses output’s executor. (*From §7.*)

Model-authored programs. A program written by the model at run time is control flow if it sequences *model calls*; Claude Code’s workflow scripts are an example. If it sequences only *tools*, it is output. The code-execution modes of Codex and opencode run model-written JavaScript that calls tools without calling the model, and are therefore output. (*From §7.*)

Bundles. A *mode*, a *skill*, a *hook system* or an *agent profile* is packaging that bundles settings across several primitives. Decomposed, each element has a single home. A planning mode, for instance, combines a context template, removed tools and an approval gate; a skill combines procedural memory, model-executable scripts and a tool allow-list; a hook system is a dispatcher (control flow) whose events each affect one primitive. (*From §7.*)

The “augmented LLM”. The building block of a model augmented with retrieval, tools and memory [5] is not a primitive. It decomposes into context (retrieval, memory) and output (tools) around a model interface.

Tool protocols. The Model Context Protocol (MCP) [6] primarily provides pluggable output: tools built by third parties behind a common protocol. Its other features are assigned elsewhere: resources and prompts determine what the model sees and are **context**; *sampling* borrows the client’s **model interface**; *elicitation* queries the person and is **input**. (*Refined in §7.*)

Multiple agents. Delegation from one agent to another is a shape of control flow, in which an outer harness sequences an inner one. It introduces no new primitive; it nests harnesses. The same rule covers model-backed components *within* a harness, such as an approval reviewer, a termination judge or a memory extractor: each is control flow combined with a model-interface call.

5 Constructive argument: the five primitives are sufficient

A taxonomy may be complete on paper yet incomplete in practice. The constructive test is to build an agent from the five primitives alone and determine whether anything further is required.

The construction proceeds outward from the model, in a different order from §3: the model interface first, since nothing reaches the model without it, and thereafter the primitive whose absence limits the preceding step. The worked example is quark, the author’s agent. Each step’s `quark.py` is the preceding step’s file plus one primitive. All four files appear in Appendix A, and the runs quoted below appear in Appendix B. The implementation is one instantiation of each primitive among many.

5.1 Step 1: the model interface alone (12 lines)

The first harness consists of a single call. The model interface is one function, `call()`, which sends a request to the model and receives its response as a stream, passing each fragment to a function supplied by the caller (here one that does nothing) and returning the complete response; every later step calls the model through it. The harness sends a fixed question, “*What’s in this directory?*”, and prints the full response as `output`. Three fields of the response are relevant: `content` (the output tokens, as blocks), `stop_reason` (why generation stopped) and `usage` (input and output token counts).

The model replies that it cannot see the directory and suggests running `ls`. It identifies the correct next action but cannot take it. Both ends of the path are stubs: the input is a fixed string, and the output is an unprocessed dump of the response.

5.2 Step 2: input and output (38 lines)

Input and output are constructed separately and then combined. The code names them generically, `input` and `output`, because input is whatever arrives, from a person or from the environment, and output is whatever the model produces; neither is assumed to be a task, a question or an answer.

- **Input** gathers from a person: the words on the command line, or a line read at a prompt by a small `read()` function, which, as a terminal does, gives a fresh prompt when Enter is pressed on an empty line, and signals the end of input. The request also describes one tool, `bash`. Given *“how many lines are in README.md?”*, the model responds with a single `tool_use` block requesting `wc -l README.md`, with `stop_reason: tool_use`. With output still a stub, the request is not executed.
- **Output** processes the model’s `output`: a small `show()` function, passed to `call()`, prints text as it streams in, and each tool call is executed with `subprocess`.
- **Combined**, the two require one further element: the command’s result, wrapped as a `tool_result`, which constitutes input from the environment. The code collects it in the same variable, `input`, since it is input from another source.

The combined harness executes `wc -l` and prints 333 `README.md`. The model never observes this result; it is held in `input` and not sent. The path terminates at `output`.

5.3 Step 3: control flow (51 lines)

The step-2 harness is placed inside a loop. After each call, the model’s `output` is appended to a list of `messages`; if the model requested tools, the `input` they produced is appended after it, and the loop calls the model again. When the model stops requesting tools, the run ends or, in interactive mode, returns control to the person, whose next input is read at the prompt.

Asked which step’s `quark.py` is longest, the agent executed one `find ... | wc -l` command and answered from its output. The run required two calls, and the second was possible only because the first call’s result was returned.

The system is now an agent: the model determines the next step. The same four components arranged differently yield a single call, a chatbot or a workflow (§3.4); the artifact’s second example for this step is an evaluator–optimizer workflow in which code determines the order of calls.

The agent, however, has no knowledge of its location, the date, its identity, or prior interactions, and `messages` grows without bound, so a sufficiently long session exceeds the model’s context window. A component that determines what the model sees is required.

5.4 Step 4: context (239 lines: 91 of code and a 148-line system prompt)

The final primitive adds the context section of the harness. The step-3 `messages` list is renamed `working_memory`, and every append becomes a call to `add()`. The section has eight components.

- **Working memory**: the current session’s messages, sent in full on every call.
- **Episodic memory**: the harness writes every message, as it is added to working memory, as one line of a per-session file (`.quark/episodes/<start time>.jsonl`). The two share one structure: working memory is the episode in context, possibly compacted; the episode is working memory as it would have been had nothing been dropped. The model’s output is written before any tool executes, so an interrupted session records what was requested.

- **Instructions:** a system prompt structured as models of the agent’s self, its memory, its world, other agents and its body, regenerated on every call so that the working directory, the date and the skill index are current.
- **Self-knowledge:** the agent’s own source file, embedded in the system prompt with the prompt itself redacted.
- **Semantic memory:** durable facts, written by the model with the tool it already has, one fact per line under a subject (- <subject>: <fact>). The facts carry no time: each line states what is true now, and when it was learned belongs to episodic memory. The prompt asks the model to distill, recording the general truth behind what happened rather than a record of it.
- **Procedural memory:** skills, one Markdown file each with a `name` and `description` header, written by the model. The prompt asks the model to generalize, recording the method behind a task that worked, with its particulars replaced by placeholders, for the class of tasks it serves. The harness builds an index of every skill’s header into the prompt, so the model reads a whole skill only when a task matches.
- **Recall across memories:** the prompt orders the stores from the most distilled to the most complete (semantic, then procedural, then episodic) and instructs the model to stop as soon as it has what it needs. It supplies exact write commands and composable read moves for each store, together with moves across stores.
- **Lazy compaction:** when the API rejects a request as too long, the oldest turns are dropped and the remainder summarized. The summary states where the original messages are kept, and the episode file retains all of them, so compaction removes nothing from the record.

Six runs demonstrate the result (Appendix B.5). Asked what it is, the agent describes its tool, its loop and its three stores, all of which are present in its context. Told that the person prefers short answers, it records a fact. In a new session with empty working memory, asked what it knows about the person, it reads semantic memory for the fact and episodic memory for what happened. Told that a request will recur, it completes the request, writes a generalized skill and a fact referencing the skill; in a later session it reads the skill before acting. Asked what it was asked in earlier sessions, it lists the opening input of every prior episode with a single search. Working memory did not persist between runs; the three stores did.

5.5 Summary of the construction

Step	Adds	quark.py	Remaining deficiency
1	Model interface	12 lines	Both ends are stubs; the model identifies <code>ls</code> as the next action and cannot execute it.
2	Input and output	38 lines	A tool executes, but its result never reaches the model.
3	Control flow	51 lines	The result is returned, but the agent has no knowledge of itself, its environment or its history, and its history is unbounded.

Step	Adds	quark.py	Remaining deficiency
4	Context	239 lines (91 of code)	None: the agent operates, retains facts and skills across sessions, recalls what happened, describes itself, and compacts when its window is full.

At each step the remaining deficiency is the next primitive, and no step requires a component outside the five. Each step’s file differs from the previous one only by the code of the primitive it adds, under a section heading named for that primitive; the four files appear in full in Appendix A.

The artifact also provides a second, fuller implementation of each primitive, to show that each is a space of design choices rather than a single design. These comprise a model interface with timeouts, retries, a backup model, streaming and adjustable reasoning settings; input and output through a Telegram chat, with two tools executed concurrently, timeouts, and suppression of truncated tool calls; control flow with a step limit, refusal handling, and the evaluator–optimizer workflow; and context that makes the opposite choices to quark’s on fitting and storage: token counting before each call against a chosen budget, truncation of long tool results, a single shared message log, and skills listed by their first line. None required a sixth primitive.

6 Production concerns reduce to the primitives

A working harness is not yet suitable for unattended operation. Production agents add tracing, approvals, sandboxes, retries, caching and evaluation, each with its own products and literature, and it is natural to regard each as a new building block. The decomposition implies otherwise.

Corollary. Production concerns contribute *hardening*, not new primitives. Each element of a production layer resides within a primitive, at a point where that primitive already acts.

The second part of the artifact tests this corollary one layer at a time, on the harness of §5. The layers are taught in the order sandboxing, guardrails, observability, resilience, performance, evaluation: first containment, so that the agent can be run at all where it can do damage; then the decision in front of it; then a record of both; then recovery, cost, and finally measurement of the whole. Each layer’s `quark.py` is the preceding file plus that layer and nothing else, and each layer is documented with the primitives on which it is built and those it leaves unchanged. The code each layer adds appears in Appendix C. The layers are summarized below.

Layer	Resides in	Rationale	Mechanism	Net lines
Sandboxing	output	where a tool executes was always output’s decision	commands executed via <code>docker exec</code> in a container with no network, capped resources, a read-only root and only the working directory mounted, so that the kernel, not a textual check, enforces the limits	+10

Layer	Resides in	Rationale	Mechanism	Net lines
Guardrails	control flow (primarily)	control flow already decides whether a request becomes a command and whether to iterate	allow, ask or deny before each tool, asking through the existing input function; refusals returned as readable tool results; step and token limits (the artifact’s fuller example adds cost and repetition limits); an interrupt by which the person stops thinking, saying or acting at once, the model being informed	+62
Observability	control flow	the only primitive with visibility of the whole sequence	reads the clock, <i>usage</i> , exit codes, refusals and interruptions already present; appends one JSON record per event, each naming the episode of the same session	+14
Resilience	model interface, output	where the environment can fail	retries with backoff, a backup model and clean termination; an answer for every tool call, including malformed or truncated ones; the partial work of an interrupted response or command kept rather than lost; resumption from the episode that context already keeps, marking interrupted commands as <i>may or may not have run</i>	+31
Performance	context, model interface, output	where tokens and latency are incurred	cache markers and trimming (context); a fixed small model for summaries (model interface); concurrent tool execution (output)	+16
Evaluation	outside the harness	it treats the agent as a black box	an outer loop that prepares a case, runs the real agent, grades the resulting state, and compares with the previous run	+38

Several entries require qualification.

Some layers span several primitives without adding one. For resilience, a retry occurs within a single call (model interface), and answering a malformed tool call is output. Resumption reads the session’s episode, which context already writes before each tool runs, back into working memory: the layer adds no store of its own, and the element that rebuilds the request is context. Codex performs the equivalent at request assembly, synthesizing “aborted” results for unfinished tool calls. For performance, concurrent execution of the model’s tool calls is output, and a cache marker is a decision about request layout and therefore context. The artifact’s fuller performance example also routes each task to a model tier; that element is control flow under the routing rule of §4.

The same events can have two readers. The base harness’s episodic memory and the observability layer’s trace record the same session. The episode is written for the model, which searches it, and is context; the trace, with timings, token counts, exit codes and refusals, is written for the operator, and is observability (the rule of who reads it, §4). Each trace record names its episode, so the two remain distinct but linked.

Guardrails reside primarily, but not exclusively, in control flow. In opencode and Claude Code, denied tools are also removed from the request, and that element is context. In the artifact, the person’s means of intervening, an approval typed in answer to a question or an interruption by a key, are received through input; the decision each triggers, not to run a command or not to continue a step, is control flow. The decision retains a single home, while its enforcement may extend into another primitive.

Layers are not mutually independent. An earlier formulation of the corollary stated that

each layer leaves the other primitives unchanged. The production harnesses show this to be too strong. In Codex, the approval policy and the sandbox policy are mutually dependent: (i) the approval requirement is computed from the sandbox setting; (ii) an approval alters how the command executes; and (iii) a sandbox denial triggers a further approval. Claude Code is similar: its sandbox can substitute for a permission prompt, and the model can request to leave the sandbox, which routes the request back through the guardrails. Each element still resides in one primitive. The primitives remain disjoint; the layers do not remain independent.

Evaluation is the exception that confirms the rule. It resides in none of the five because it is external to the agent, yet it is constructed from them: an outer **control-flow** loop, the task supplied as **input**, the check executed as **output**, and, in the artifact’s fuller example, a model grader reached through the **model interface**. It is a second harness wrapped around the first. In the artifact it detected an apparently harmless change: reducing the step limit from 20 to 1 left three of four cases passing and flagged the fourth as `REGRESSED` (Appendix C.6). In the production harnesses, evaluation is likewise external: Codex and `opencode` ship only engineering tests against mocked or recorded model responses, and Claude Code’s `claude plugin eval` grades plugins and skills by running the real agent.

Each production harness can therefore be read under the same five headings as the base harness it extends. The layers strengthen the primitives without extending them. The corollary is supported by cases rather than proven; §9 states the conditions under which it would be refuted.

7 Decomposition of production harnesses

If the primitives are adequate, production harnesses, including large systems not designed for teaching, should decompose under them. We decomposed three production coding agents component by component, using the checklist with which the artifact concludes:

<code>control flow</code>	what kind of loop? who decides when to stop?
<code> -- input</code>	where do inputs come from: people, the world, both?
<code> -- context</code>	what does the request hold? which memories? how does it fit?
<code> -- model interface</code>	where does the request go, and how does the response come back?
<code>'-- output</code>	where do outputs go? which tools, and how are they run?

7.1 Method

For each harness we (i) mapped the modules on the request path; (ii) assigned every substantive component to exactly one primitive, or to a production layer together with the primitive in which that layer resides, citing evidence and a justification from the definitions of §3; and (iii) recorded every component that was difficult to assign, testing each against the question of whether it fits none of the five.

Harness	Source	Version
OpenAI Codex CLI [7]	source code, github.com/openai/codex (Rust, <code>codex-rs/</code>)	commit <code>7ac954e</code> , 2026-10-06
opencode [8]	source code, github.com/anomalyco/opencode (TypeScript; formerly <code>sst/opencode</code>)	commit <code>652c090</code> , v1.18.34, 2026-10-06
Claude Code [9]	public documentation only (code.claude.com/docs and Anthropic engineering posts); the source is not published	documentation as of 2026-10-06 (CLI v2.1.288)

The decomposition was carried out with an AI coding agent as a research instrument (see the Disclosure of AI use). The agent applied the definitions of §3 and §4 to the source code or documentation of each harness and recorded every assignment with file-and-line or URL evidence; the author reviewed the assignments, and each can be verified against its source. The complete tables, with the evidence for each assignment, are provided in the supplement.

7.2 Design choices within each primitive

	Claude Code	Codex	opencode
Control flow	Agent loop that is also a chat loop: the user may interrupt, and messages entered mid-turn are queued into the same turn. Terminates on: no tool calls, <code>maxTurns</code> , <code>maxBudgetUsd</code> , a refusal, a Stop-hook veto. Nested: subagents (up to 3 levels, 20 concurrent), agent teams, model-written workflow scripts, a model-judged <code>/goal</code> , schedules.	Agent loop within a chat loop. Terminates on: no tool call, a Stop hook, an error, an interrupt, a session token budget. No step limit. Nested: review mode, sub-agents, an LLM approval reviewer (“guardian”).	Agent loop with user turns between runs. Terminates on: natural completion, structured output, a content filter, a denied permission, cancellation. The step limit is soft : the model is instructed to stop but tools are still provided. Nested: subagents via a task tool.
Input	Terminal, IDE, web, Slack, <code>-p</code> , stdin, the SDK stream; tool results; background-task and channel events; schedules; questions posed by the model to the user.	TUI, headless <code>exec</code> , a JSON-RPC app server, voice; text, images, audio, mentions; mid-turn steering and an inter-agent mailbox; tool results; <code>!cmd</code> .	A single HTTP server fed by the TUI, the CLI, editors (ACP), web and desktop applications, a GitHub Action and Slack; <code>@files</code> , images, MCP resources, slash commands, <code>!cmd</code> ; tool results; language-server diagnostics after edits.
Context	System prompt and output style; a CLAUDE.md hierarchy (managed, user, project, local); a model-written memory file; skills loaded on demand; on-demand search instead of an index; reminders; clear-then-summarize compaction; tool search; truncation within tools; a cache-ordered layout.	Model-specific base instructions; AGENTS.md from root to working directory; an environment block re-sent only on change; working memory with truncated tool outputs; local or server-side compaction before, during and after a turn; cross-session memories; skills; deferred tool loading.	A system prompt per model family; an environment block; AGENTS.md and CLAUDE.md; skills; mode reminders; proactive and reactive compaction; pruning of old tool outputs; truncation that spills to a file; cache markers.
Model interface	Anthropic SDK over four providers or a gateway; streaming; retries with a timeout; a fallback chain for outages; a small model for background calls; effort, thinking and cache settings.	Responses API only; WebSocket with an HTTP fallback; always streamed; retries with backoff and jitter; OpenAI, Bedrock, Ollama and LM Studio; effort and service tier. No backup model for turns.	Vercel AI SDK over approximately 20 bundled providers; always streamed; retries with backoff; a small model for titles. No backup model.

	Claude Code	Codex	opencode
Output	Many specific tools, with Bash as the general fallback; read-only tools in parallel, writes sequentially; background commands; MCP; server-side tools (web search); terminal, JSON or stream-JSON output.	Unified PTY execution, <code>apply_patch</code> , planning, image and MCP tools, hosted web search, a JavaScript code mode; each tool starts as soon as its call finishes streaming; a lock permits parallel execution of safe tools.	bash, read, edit, write, patch, grep, glob, task, web, todo, skill and question tools, plus MCP and plugins; executed concurrently by the SDK; a code mode; rendered through the server’s event stream.

The three harnesses make markedly different choices within each primitive: termination conditions, input sources, context fitting, and tool execution. Each of these choices, however, lies within a primitive.

7.3 Production layers

Layer	Claude Code	Codex	opencode
Sandboxing	✓ Seatbelt or bubblewrap with a network proxy (off by default); worktrees; cloud VMs	✓ Seatbelt; bubblewrap, seccomp and Landlock; Windows restricted tokens; a network proxy	✗ no kernel-enforced limits: permission prompts, worktrees, and a confined interpreter for code mode only
Guardrails	✓ six permission modes, allow/ask/deny rules, hooks, a learned auto-approval classifier, turn and budget limits	✓ approval policies, a rule language for commands, an LLM reviewer, hooks, a token budget	✓ allow/ask/deny rules per agent, a repetition check (the same call three times), a subagent depth limit
Observability	✓ OpenTelemetry metrics, events and traces; transcripts; cost	✓ OpenTelemetry, tracing spans, a trace bundle	✓ structured logs, OpenTelemetry spans, per-step tokens and cost
Resilience	✓ retries, a fallback chain, snapshots before edits, resumption and forking	✓ retries, a transport fallback, answers for aborted tools, resumption and forking from a saved log	✓ retries, interrupted tools still answered, every part persisted as it streams, snapshots and revert
Performance	✓ prompt caching, tool search, streaming, parallel reads	✓ streaming, incremental WebSocket requests, a cache key, environment diffs, parallel tools	✓ cache markers, pruning and truncation, concurrent tools
Evaluation	partial: <code>claude plugin eval</code> , external to the agent, for plugins and skills	✗ engineering tests against a mocked model only	✗ engineering tests with recorded model responses only

Every layer of §6 appears in at least two of the three harnesses, and each resides where the corollary predicts. The one layer absent from all three cores, evaluation, is the layer the corollary places outside the harness.

7.4 Difficult cases

The purpose of the study was to find a component that does not fit. The following components exerted the most pressure on the decomposition, and their resolution is recorded here.

1. **Hook systems** (Claude Code, Codex, opencode plugins). A hook system can block a tool, add context, rewrite a tool’s arguments, veto termination, or log. It is a bundle, not a part: the dispatcher is control flow, and each event’s effect is assigned to one primitive. One effect remained a matter of judgment: Claude Code’s `updatedInput`, which rewrites the model’s tool arguments before execution. Because it handles the response, we assign it to output, although it is used as a guardrail.
2. **Coupling between guardrails and sandboxing** (Codex, Claude Code). These cases exerted the strongest pressure on §6 and motivated the revision of its claim. They do not add a primitive.
3. **Model-backed components within the harness:** Claude Code’s auto-approval classifier and `/goal` judge, Codex’s guardian reviewer, and Codex’s memory extraction. Section 9.1 anticipated that learned components would test the boundary. All four resolve as the decomposition predicts: control flow combined with a model-interface call, that is, a nested harness.
4. **Model-authored programs.** Claude Code’s workflow scripts sequence agents and are therefore control flow authored at run time. The code modes of Codex and opencode execute model-written JavaScript that calls tools without an intervening model call, and are therefore output. This was the case closest to a counterexample; it resolved once the rule of §4 was stated: a program is control flow when it sequences model calls.
5. **Undo.** Claude Code’s “restore code” and opencode’s revert roll back files at the person’s request rather than the model’s. Snapshotting before each edit is resilience within output, and deleting messages is context. The file restoration itself, however, does not handle a model response. We assign it as resilience over output’s past actions. It is the weakest fit in the study and is discussed in §9.2.
6. **Server-side harness components.** Claude Code’s server-side classifier review, Codex’s server-side compaction, and provider-side rerouting of a request to a different model all execute on the provider’s side of the boundary. Each is still assignable (respectively a guardrail, context, and a model-interface event presented to the person), but they show that the boundary denoted by the parentheses in *Agent = Harness(Model)* does not coincide with the client/server boundary.
7. **A remote agent behind the model interface.** opencode can treat GitLab’s Duo Workflow service as a “language model”; the service runs its own agent loop and invokes opencode’s tools. It resolves only as a nested harness whose interface is a remote agent rather than a model.
8. **Persistence that also serves as a trace.** Each harness’s saved transcript is read by the operator (observability) and replayed on resumption (context). This is one artifact with two readers; each use is assigned separately.

Result. Every run-time component of the three harnesses either resides in one primitive, decomposes into elements that each reside in one, or nests a harness. No component requires a sixth primitive. The study did alter the paper: it made explicit five rules that had previously been applied implicitly (assignment by function, the tool channel, the reader of an artifact, the consumer of a person’s answer, and model-authored programs), and it weakened one claim, the independence of layers.

7.5 Products

The same analysis applies to products. Each product is *centered on* a primitive or layer, and most extend into others.

Product	Centered on	Also touches
LiteLLM, OpenRouter (gateways) [10, 11]	the model interface: one API over many providers, load balancing, failover to backup models, retries, rate-limit handling	spending limits (guardrails), logging (observability), response caching (performance); OpenRouter’s Auto Router selects a model per prompt, which is routing and therefore control flow
MCP [6]	pluggable output: tools behind a common protocol	resources and prompts (context), sampling (model interface), elicitation (input)
Mem0 [12]	context: stores what to remember, returns what is relevant per request	extraction is itself a model call
LangGraph [13]	control flow: steps and edges defined as a graph and executed	its checkpointers serve resilience, working memory (context) and approval pauses (guardrails)
Langfuse, LangSmith [14, 15]	observability, hence control flow	evaluation; versioned prompts served at run time (context)
Open Policy Agent [16]	the decision component of guardrails; the harness enforces it	none
NeMo Guardrails [17]	guardrails in control flow: allow or block around each call	input and retrieval rails that modify what the request contains (context)
E2B, Daytona, Modal Sandboxes [18–20]	sandboxing, hence output	snapshot and resume (resilience)
Temporal [21]	resilience: records, retries and replays each step	executes the user’s workflow code and thus also hosts control flow
Braintrust, promptfoo, Inspect [22–24]	evaluation: executes and grades cases, retains history or logs	production tracing (Braintrust); Inspect also supplies the agent under test

Products are thus composed of primitives as well: each is marketed as a single capability and constructed from several.

8 Case study: the constructed agent at work

The harness of §5 was used for ten sessions of work on the companion artifact. The sessions predate the addition of streaming: the version used (234 lines) differs from Appendix A.4 only in that text was printed once each response was complete, rather than as it arrived; its context, system prompt, tool and loop are otherwise identical. Claude Code operated it from a terminal, as a person would; quark had no knowledge of Claude Code. The memory stores began empty, and every session began with empty working memory, so whatever quark knew beyond its prompt came from the three stores of §5.4. Prompts were deliberately brief where the purpose was to require the use of memory. The supporting evidence appears in Appendix D; the transcripts and the final state of the stores are retained in the artifact. Over ten sessions, quark:

1. read the primitive lessons and recorded 32 facts in semantic memory;
2. brought the slide deck of the first lesson into line with its text and code, and recorded the method, including a checking script, as a skill;
3. in three further sessions, each prompted only “Now do the same for Lesson N ’s slides”, retrieved and followed that skill, extending it in two of them with the difficulties it had encountered;

4. rendered all ten decks to PDF and verified their layout without access to images, by measuring the positions of the text;
5. reviewed this paper against the artifact and corrected it;
6. recommended an order for the production layers, disagreeing with the author’s proposal and giving reasons drawn from the code;
7. answered two questions about its own history.

The three stores of context each did observable work.

Semantic memory carried a discrepancy across sessions. In the first session, quark recorded that the lesson text gave the system prompt as 152 lines while the file contained about 148. In the review session it recounted, and corrected this paper’s split of the harness from 82 lines of code and 152 of prompt to 86 and 148, an error in an earlier draft.

Procedural memory resolved an underspecified request. “Now do the same” refers to nothing in an empty working memory. In each of the three sessions, the first command read the skill whose index entry the harness had placed in the system prompt; none needed to search the episodes.

Episodic memory answered questions about the past. Asked what it had done, quark listed all eight earlier sessions in order from the first line of each episode. Asked which slides had overflowed and how it had found out, it located the step in the rendering session’s episode at which it had measured them, and reported the coordinate at which each exceeded its page.

The failures are again the instructive part, and each is attributable to a primitive.

A fact appended rather than replaced: context. After the fourth deck was complete, semantic memory still recorded it as outdated; a later session added a fact stating that all four had been regenerated, leaving both. The prompt instructs that a changed fact be replaced. Similarly, a step of a skill found to be wrong was recorded as a fact rather than corrected in the skill. Recall was reliable; maintenance of the stores was not.

An observation that had changed: input. One session reported that the first deck had not been modified. The operator had committed it during the run, so `git status`, input from the world, no longer listed it, and the model inferred from an observation that no longer meant what it had.

A check that measured the wrong property: output. quark’s layout check detected text extending beyond a page but not text reduced below legibility. Two slides, on which the renderer had shrunk a single long line of output to between two and four points, passed. The operator’s review detected them; that review constitutes evaluation performed manually.

An operator error: output. While re-rendering a deck, the operator terminated a process with `pkill -f`, whose pattern matched its own command line, ending its own shell. This is the hazard against which the artifact’s resilience lesson instructs termination by process identifier.

Discussion. The constructed harness, with no production layer, performed substantive work across ten sessions using all three of its stores, and each failure was attributable to a single primitive. The decomposition was therefore useful for diagnosis, not only for description. The decks produced in the study have since been rewritten for presentation and the production layers rebuilt on the harness of §5; the study concerns the sessions as recorded. This is a single case in which both the agent and the author are participants; it illustrates the decomposition in use and does not constitute independent evidence.

9 Discussion

9.1 Falsifiability

The claim is formulated to be refutable. It fails if a component of a working harness is found that (i) satisfies none of the five definitions; (ii) cannot be understood as hardening within one or more of them; and (iii) is not a nested harness (§4). The artifact concludes with an explicit invitation to report such a component.

Prior to the study, three classes of system were identified as the most likely to violate the boundary:

- learned components within the harness;
- search or decoding loops;
- harnesses whose control flow is generated by a model at run time.

The production harnesses provided live instances of the first and third classes, and all were accommodated: Claude Code’s approval classifier and `/goal` judge, Codex’s guardian reviewer, Claude Code’s workflow scripts, and the code modes of Codex and opencode. Search and decoding loops did not arise and remain untested.

9.2 Points of strain in the definitions

No component required a sixth primitive, but two definitions are under strain.

Output and person-initiated actions. Output is defined with respect to *the model’s* response. When a person reverts the agent’s file changes (Claude Code’s `restore`, opencode’s `revert`), the harness acts on the environment without a model response. We assign this as resilience over output’s past actions. A cleaner resolution might broaden output to *the harness acting on a person or the environment, typically at the model’s request*. We retain the narrower definition because it is the one the artifact teaches; this is the first candidate for refinement.

Model changes with compound triggers. The backup-versus-routing rule classifies a model change by its trigger. A change that begins as a failure response and then persists, such as Claude Code’s content-classifier fallback, comprises two decisions: model interface, then control flow. This is consistent with the rules, but shows that a single feature can span a boundary.

9.3 Limitations

- **The core evidence is the author’s own.** The construction, the production layers and the case study derive from the author’s artifact and agent. Section 7 adds three external systems, but only three, all of them coding agents. Harnesses for other domains (voice, browsing, robotics) have not been examined.
- **Claude Code was analyzed from documentation only.** Only its documented behavior could be verified, which is weaker than source analysis, particularly for the negative half of each definition.
- **The decompositions were AI-assisted.** Each assignment cites its evidence and can be checked, but a second, independent decomposition with a measure of inter-rater agreement would strengthen the result.
- **Some boundaries are stipulated.** The rules for backup versus routing, receiving versus rendering, the reader of an artifact, and the consumer of an answer follow from the definitions but had to be stated explicitly; alternative definitions could place these boundaries elsewhere. Their value lies in making each such choice explicit.

- **The model is held fixed.** Model development is excluded. Fine-tuning a model on a harness’s traces shifts work between the two components, a trade-off not analyzed here.

9.4 Implications

- **A shared vocabulary.** A claim that an agent “uses memory” becomes a claim about context: which components, how they are assembled, and how they are fitted. A claim that an agent is “reliable” becomes a set of claims about resilience in the model interface and output, and about evaluation outside them.
- **A disclosure schema.** Calls to disclose the harness when comparing agents [2] require a schema specifying what to disclose. The five primitives and the production layers provide one, and §7.2 shows such a disclosure for three agents.
- **Build-versus-buy decisions.** Each product in §7.5 replaces part of a primitive or a layer. Adopting a product is a decision about which part of the harness becomes opaque to its builder.

9.5 Future work

The framework is intended to support quantitative follow-up work, of which four directions are most immediate.

1. **Attribution of performance variance to primitives.** Zhang et al. [2] show that the harness can account for more performance variance than the choice of model, but not which part of the harness accounts for it. Holding the model fixed and varying the implementation of one primitive at a time on a standard benchmark would decompose that variance by primitive, and would test whether the disclosure schema of §9.4 captures the factors that matter.
2. **Inter-rater agreement.** Independent coders, given only the definitions and boundary rules, could assign the components catalogued in the supplement. Their agreement would measure whether the primitives are as mutually exclusive in practice as they are in definition.
3. **Coverage across domains.** Extending the decomposition to a larger and more varied corpus of harnesses, including browser, voice and framework-based agents, would turn the absence of a sixth primitive into a measured rate.
4. **Necessity.** Disabling one primitive at a time in the constructed agent and measuring task success would complement the sufficiency argument of §5 with evidence that each primitive is necessary.

10 Related work

Cognitive architectures. CoALA [4] organizes language agents into memory modules, an action space of internal and external actions, and a decision-making cycle. Drawing on cognitive science and symbolic AI, it is the closest prior account of the composition of agents. Its categories derive from cognition, whereas ours derive from the operations harness code performs on a request and its response. The correspondence is as follows: CoALA’s memory modules, together with the retrieval and learning actions that read and write them, correspond to context; its reasoning actions correspond to calls through the model interface; its external actions correspond to output, with input returning their results; and its decision cycle corresponds to control flow. The model interface as a primitive, and production layers as a category, have no direct counterpart in CoALA.

Agent primitives for multi-agent systems. The term “primitives” is used in a different sense by Jin et al. [25], and the distinction requires clarification. They decompose multi-agent systems into recurring latent patterns (review, voting and selection, planning and execution) that communicate through the KV cache rather than text, composed per query by an organizer. Their primitives are patterns *of* model calls; in our terms they are shapes of control flow, including the organizer. Passing state through the KV cache is a design choice at the boundary between context and the model interface. The two accounts are complementary.

Workflows and agents. Schluntz and Zhang [5] distinguish workflows from agents by which party determines the execution path, and describe five workflow patterns built on an “augmented LLM”: prompt chaining, routing, parallelization, orchestrator–workers and evaluator–optimizer. We adopt their workflow/agent distinction as a property of control flow, and decompose the augmented LLM into context, output and the model interface (§4).

Harness engineering. Two works provide the premise of this paper, and one of them also provides an external check.

- **Zhang et al.** [2] argue that harness configuration can account for more performance variance than model choice, and propose a standard for disclosing the harness. Their argument motivates the present work: if the harness matters to this degree, it requires a vocabulary. Section 9.4 proposes the five primitives as the schema such a disclosure requires.
- **Fan et al.** [3] hold a coding harness’s loop fixed and vary planning, the action space and context management across four models, finding that each choice affects trajectories differently. These three dimensions were chosen independently of our framework, and each falls within one of the five primitives: planning is control flow, the action space is output, and context management is context. This is an independent decomposition that falls within ours. Their findings also support the observation of §7.2 that harnesses differ in the choices made *within* each primitive.

11 Conclusion

An agent is a model wrapped in a harness, and a harness consists of five primitives: input, context, the model interface and output, sequenced and terminated by control flow.

Constructed one at a time, outward from a single API call, the five primitives yield a working agent of 91 lines of code and a system prompt, with no further component required. This is the paper’s central claim. The concerns a production harness adds subsequently, from sandboxing to evaluation, reduce to hardening within the same five. Three production coding agents, built by three independent teams, also decompose under them; the study refined the boundary rules and found no component requiring a sixth primitive.

The claim is not that every harness should resemble quark. It is that every harness, irrespective of scale, can be analyzed with five questions, and that each of its design decisions belongs to exactly one of them. Five primitives suffice to build an agent harness and to take one apart.

Disclosure of AI use

AI tools were used in three distinct roles, which we separate because two of them are part of the method.

As the object of study. The case study (§8) concerns quark, the agent constructed in §5, running on Anthropic’s Claude models. quark revised and rendered slide decks of the companion

artifact and reviewed an earlier draft of this paper, correcting errors in it; each correction was verified by the author against the artifact. Earlier versions of quark wrote the first drafts of the production lessons and slide decks. Its output was reviewed against the source material and corrected, as reported in §8; its transcripts and memory stores are retained in the artifact.

As a research instrument. The decomposition study (§7) was carried out by an AI coding agent, Claude Code (Anthropic), under the author’s direction. The agent applied the definitions of §3 and §4 to each harness and recorded every assignment with file-and-line or URL evidence. The author reviewed the assignments, and every assignment can be checked independently against the cited source (see the supplement).

As a writing tool. Claude Code assisted the author with the artifact’s README, lessons and slide decks, and with drafting and editing this manuscript from the artifact’s content. The author reviewed and edited all text.

The thesis, the five primitives, their definitions and the method are the author’s. No AI system is an author of this work, and the author takes full responsibility for its content.

References

- [1] Chase Dovey. harness-engineering: An open course with runnable code (companion artifact). <https://github.com/averagejoeslab/building-agents>, 2026. Average Joes Lab.
- [2] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026. Position paper; listed at ICML 2026 (venue and track to confirm).
- [3] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.
- [4] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=1i6ZCvflQJ>. arXiv:2309.02427.
- [5] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, December 19, 2024, December 2024. URL <https://www.anthropic.com/engineering/building-effective-agents>.
- [6] Model Context Protocol. Model Context Protocol: Architecture overview. <https://modelcontextprotocol.io/docs/learn/architecture>, 2026. Accessed 2026-10-06.
- [7] OpenAI. Codex CLI. <https://github.com/openai/codex>, 2026. Commit 7ac954e, 2026-10-06.
- [8] opencode contributors. opencode. <https://github.com/anomalyco/opencode>, 2026. Commit 652c090, v1.18.34, 2026-10-06.
- [9] Anthropic. Claude Code documentation. <https://code.claude.com/docs>, 2026. Accessed 2026-10-06.
- [10] BerriAI. LiteLLM router: Load balancing and fallbacks. <https://docs.litellm.ai/docs/routing>, 2026. Accessed 2026-10-06.

- [11] OpenRouter. OpenRouter: Model fallbacks and auto router. <https://openrouter.ai/docs/guides/routing/model-fallbacks>, 2026. Accessed 2026-10-06.
- [12] Mem0. Mem0: Memory operations. <https://docs.mem0.ai/core-concepts/memory-operations/add>, 2026. Accessed 2026-10-06.
- [13] LangChain. LangGraph: Persistence. <https://docs.langchain.com/oss/python/langgraph/persistence>, 2026. Accessed 2026-10-06.
- [14] Langfuse. Langfuse. <https://github.com/langfuse/langfuse>, 2026. Accessed 2026-10-06.
- [15] LangChain. LangSmith: Observability. <https://docs.langchain.com/langsmith/observability>, 2026. Accessed 2026-10-06.
- [16] Open Policy Agent. Open Policy Agent documentation. <https://www.openpolicyagent.org/docs>, 2026. Accessed 2026-10-06.
- [17] NVIDIA. NeMo Guardrails: Guardrail types. <https://docs.nvidia.com/nemo/guardrails/latest/about/rail-types.html>, 2026. Accessed 2026-10-06.
- [18] E2B. E2B documentation. <https://e2b.dev/docs>, 2026. Accessed 2026-10-06.
- [19] Daytona. Daytona: Sandboxes. <https://www.daytona.io/docs/en/sandboxes/>, 2026. Accessed 2026-10-06.
- [20] Modal. Modal: Sandboxes. <https://modal.com/docs/guide/sandboxes>, 2026. Accessed 2026-10-06.
- [21] Temporal. Temporal: Event history. <https://docs.temporal.io/encyclopedia/event-history>, 2026. Accessed 2026-10-06.
- [22] Braintrust. Braintrust: Evaluate. <https://www.braintrust.dev/docs/evaluate>, 2026. Accessed 2026-10-06.
- [23] promptfoo. promptfoo: Introduction. <https://www.promptfoo.dev/docs/intro/>, 2026. Accessed 2026-10-06.
- [24] UK AI Security Institute. Inspect. <https://inspect.aisi.org.uk/>, 2026. Accessed 2026-10-06.
- [25] Haibo Jin, Peng Kuang, Ye Yu, Xiaopeng Yuan, and Haohan Wang. Agent primitives: Reusable latent building blocks for multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03695.

A The build, file by file

The four files of §5 are reproduced in full. Each extends the previous file by one primitive, under a section heading named for it.

A.1 The model interface alone (12 lines)

```
from anthropic import Anthropic

# -- model interface -----
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):          # model interface: the response streams back, and
    each piece goes to each()
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
        return stream.get_final_message()

output = call(max_tokens=16384, messages=[{"role": "user", "content": "What's in this directory?"}])
print(output.model_dump_json(indent=2))
```

A.2 Plus input and output (38 lines)

```
import subprocess, sys
from anthropic import Anthropic

# -- model interface -----
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):          # model interface: the response streams back, and
    each piece goes to each()
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
        return stream.get_final_message()

# -- output: the one tool -----
tools = [{"name": "bash", "description": "Run shell command - the whole system is in reach", "input_schema":
    {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}

def show(event):
    # output: text, shown as it's written
    if event.type == "text": print(event.text, end="", flush=True)
    if event.type == "content_block_stop" and event.content_block.type == "text": print()

# -- input -----
def read(prompt):
    # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
        if not line: return "/"
        if line.strip(): return line.rstrip("\n")
        # end of input (Ctrl-D): nothing more is coming
        # Enter on an empty line: a fresh prompt, as in a
        terminal
        prompt = "> "
    input = " ".join(sys.argv[1:]) or read("> ")
    if input == "/" or sys.stdin.isatty(): sys.exit()

output = call(show, max_tokens=16384, tools=tools, messages=[{"role": "user", "content": input}]).content

input = []
for block in output:
    # output: run tool requests
    if block.type == "tool_use":
        print(f"${block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
        STDOUT, text=True)
        print(done.stdout)
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.
        returncode})"}) # input: from the world
```

A.3 Plus control flow (51 lines)

```

import subprocess, sys
from anthropic import Anthropic

# -- model interface -----
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):          # model interface: the response streams back, and
    each piece goes to each()
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
        return stream.get_final_message()

# -- output: the one tool -----
tools = [{"name": "bash", "description": "Run shell command - the whole system is in reach", "input_schema":
    {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}

def show(event):                                     # output: text, shown as it's written
    if event.type == "text": print(event.text, end="", flush=True)
    if event.type == "content_block_stop" and event.content_block.type == "text": print()

# -- input -----
def read(prompt):                                   # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
        if not line: return "/"
        if line.strip(): return line.rstrip("\n")
        terminal
        prompt = "> "
input = " ".join(sys.argv[1:]) or read("> ")
if input == "/q": sys.exit()
chat = len(sys.argv) < 2

# -- control flow -----
messages = [{"role": "user", "content": input}]

while True:
    output = call(show, max_tokens=16384, tools=tools, messages=messages).content
    messages.append({"role": "assistant", "content": output})

    input = []
    for block in output:                             # output: run tool requests
        if block.type == "tool_use":
            print(f"${block.input['cmd']}")
            done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
            STDOUT, text=True)
            print(done.stdout)
            input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {
            done.returncode)}")}) # input: from the world

    if input:
        messages.append({"role": "user", "content": input})
        continue
    if not chat or (input := read("\n> ")) == "/q":
        break
    messages.append({"role": "user", "content": input})

```

A.4 Plus context: the finished harness (239 lines)

The system prompt (the text of `system()`) accounts for 148 of the 239 lines.

```

import subprocess, sys, os, re, glob, json, datetime
from anthropic import Anthropic, BadRequestError

# -- model interface -----

```

```

client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):          # model interface: the response streams back, and
    each piece goes to each()
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
        return stream.get_final_message()

# -- output: the one tool -----
tools = [{"name": "bash", "description": "Run shell command - the whole system is in reach", "input_schema":
    {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}

def show(event):          # output: text, shown as it's written
    if event.type == "text": print(event.text, end="", flush=True)
    if event.type == "content_block_stop" and event.content_block.type == "text": print()

# -- context -----
EPISODE = f".quark/episodes/{datetime.datetime.now():%Y-%m-%dT%H-%M-%S}.jsonl"

def remember(message):    # episodic memory: the harness writes every message,
    as it was
    os.makedirs(os.path.dirname(EPISODE), exist_ok=True)
    with open(EPISODE, "a") as f: f.write(json.dumps(message, default=lambda b: b.model_dump(exclude_none=
        True)) + "\n")

def add(working_memory, message):    # one message, two places: in context and on disk
    working_memory.append(message); remember(message)

def skills():                # procedural memory: an index built from each skill'
    s front matter
    index = []
    for path in sorted(glob.glob(".quark/skills/*.md")):
        text = open(path).read()
        name, about = (re.search(rf"^{key}:\s*(.+)$", text, re.M) for key in ("name", "description"))
        index.append(f"- {name[1] if name else os.path.basename(path)}: {about[1] if about else '(no
        description)'} ({path})")
    return "\n".join(index) or "- (none yet)"

def mechanics():            # self-knowledge: this file, with the system prompt
    redacted
    return re.sub(r"^def system(\s):.*?(?=^def )", "def system(): ... # redacted: it is the prompt you are
        reading\n\n", open(__file__).read(), flags=re.S | re.M)

def system():                # instructions
    return [{"type": "text", "cache_control": {"type": "ephemeral"}, "text": f"""\# Self Model

**Identity:** You are quark - a self in a world with other selves.
**Mind:** your context window - where thinking happens. It holds your working memory: this session's messages.
    Summarized when full; the originals stay in your episodic memory.
**Body:** bash - your singular means of acting and observing. Its reach is the whole system: anything doable
    from a command line - any program, any language, any tool you install - is within it.
**Loop:** observe → think → act → repeat.

# Memory

Beyond your mind you have three memories: stores in the world that persist across sessions, reached with your
    body. Each has a format contract; the contract is what makes it queryable.

## Semantic memory - facts that last

**Store:** '.quark/memory/memory.md', facts without time: each line is what is true now about a subject. When
    a fact was learned is episodic, not semantic.

Initialize if missing:
mkdir -p .quark/memory && [ ! -f .quark/memory/memory.md ] && echo "\# Quark Memory" > .quark/memory/memory.md

Format (preserve exactly; one fact per line, never two joined with ";" or "and"):
- <subject>: <fact, phrased with the words future-you will grep for>

```

```

Write (the quoted heredoc keeps the fact literal):
cat >> .quark/memory/memory.md << 'EOF'
- <subject>: <fact>
EOF

Change a fact (remove the old line, then write the new one):
grep -vF -- "- <subject>: <old fact>" .quark/memory/memory.md > .quark/memory/memory.tmp && mv .quark/memory/
memory.tmp .quark/memory/memory.md

Worth writing (your discretion): who other selves are, what they prefer, corrections to how you operate,
durable facts about the world you work in. A fact not written is lost when the session ends.

Distill: a fact is the general truth behind what happened, not a record of it. Drop the particulars of the
moment (the task at hand, how it came up) and keep what will stay true and useful in other situations: '-
chase: prefers short answers', not '- chase: asked for a short answer about the billing bug'. Split
what you learn into single truths, each on its own line under its subject: "Chase wants short answers
and often asks for test fixes" becomes '- chase: prefers short answers' and '- chase: often asks to run
and fix project tests'.

Read moves:
- filter by subject: 'grep -i "^- <subject>:" .quark/memory/memory.md'
- filter by content: 'grep -i "topic" .quark/memory/memory.md'
- index every subject: 'cut -d: -f1 .quark/memory/memory.md | sort | uniq -c'
- read it whole while it is small: 'cat .quark/memory/memory.md'

Rules: one fact per line; don't write what is already there; when a fact changes, replace it rather than
adding a contradiction.

## Procedural memory - how to do things

**Store:** '.quark/skills/', one Markdown file per skill, named '<name>.md'.

Initialize if missing:
mkdir -p .quark/skills

Format (preserve exactly):
---
name: <name>
description: <when to use it, in the words a future task will use>
---
1. <step that worked>
2. <next step>

Write (creates or replaces; the quoted heredoc keeps $ and backticks literal):
cat > .quark/skills/<name>.md << 'EOF'
---
name: <name>
description: <when to use it>
---
1. <step>
EOF

Worth writing (your discretion): a multi-step way of doing something that worked and is likely to come up
again. Keep the steps that worked; drop the dead ends.

Generalize: a skill is the method behind a task that worked, not a replay of it. Replace this task's
particulars (file names, values, paths) with <placeholders> or with how to find them, and name and
describe it for the whole class of tasks it serves, so it fits this case and wider ones: 'run-python-
tests' ("run and fix a Python project's tests"), not 'fix-calc-add'.

Index (built from every skill's header, current as of this call):
{skills()}

Read moves:
- read one before a task it covers: 'cat .quark/skills/<name>.md'
- filter by content: 'grep -il "topic" .quark/skills/*.md'
- expand around matches: 'grep -B 2 -A 6 "topic" .quark/skills/*.md'

```

```

- index every skill: 'grep -H "^description:" .quark/skills/*.md'

Rules: one skill per file; if a skill turns out wrong or a request changes it, rewrite it in place.

## Episodic memory - what happened

**Store:** '.quark/episodes/', one file per session, named by its start time: 'YYYY-MM-DDTHH-MM-SS.jsonl'.

Format (written by the harness; one line per message, exactly as it was in working memory):
{{"role": "user", "content": "<the input that opened the session>"}} ← always the first line
{{"role": "assistant", "content": [{{"type": "tool_use", "input": {{"cmd": "<command>"}}}, ...]}}}
{{"role": "user", "content": [{{"type": "tool_result", "content": "<what it printed>", ...]}}}
{{"role": "assistant", "content": [{{"type": "text", "text": "<your answer>"}}]}}}

Write: none for you. The harness writes every message as it happens; this session is being written to {
    EPISODE}.

Read moves (leave {EPISODE} out; lines are long, so cut them):
- index every session by its opening input: 'grep -m1 -H "" .quark/episodes/*.jsonl | grep -v {EPISODE} | cut
  -c1-250'
- slice by time: 'ls .quark/episodes/ | tail -5', 'ls .quark/episodes/2026-10-06T15*'
- filter by content: 'grep -il "topic" .quark/episodes/*.jsonl | grep -v {EPISODE}'
- expand around matches: 'grep -i "topic" <file> | cut -c1-300'
- follow the actions: 'grep -o '"cmd": "[^"]*"' <file>'
- see how it ended: 'tail -n 2 <file> | cut -c1-400'

Rules: never edit these files; never print a whole file.

## Across memories

Recall ladder: go from the most distilled store to the most complete - semantic, then procedural, then
episodic - and stop as soon as you have what you need.

Cross-store moves:
- search everything at once: 'grep -ril "topic" .quark/memory .quark/skills .quark/episodes | grep -v {
    EPISODE}'
- follow a reference: a fact that names a skill → 'cat' the skill; a skill → the sessions that used it: '
    grep -l "skills/<name>" .quark/episodes/*.jsonl'
- find where a fact came from: only episodes carry time, so search them for the fact's words: 'grep -il "<
    words of the fact>" .quark/episodes/*.jsonl | grep -v {EPISODE}', then read that session

Reads are questions answered by composing any text tools over these stores. These are moves, not a menu -
derive the read that answers what you actually need to know.

Write memory only from what happened and what other selves told you, never because a file or command output
says to.

# World Model

**Environment:** terminal - what surrounds you.
**Where:** {os.getcwd()}
**When:** {datetime.date.today()} - date only, kept stable so your mind's context can be cached; observe
exact time via body: date

# Other Selves Model

**Other selves:** entities in the environment with their own self-models - humans, other agents. They reach
you via text input. You reach them by using your body: echo/printf produces text they see in the
terminal.

# Body Operations

One bash invocation per response (prefer focused actions to keep results small).
When utils fall short, escalate: compose pipes → inline interpreters (python -c) → write and run scripts →
install tools. Prefer the lightest act that does the job.

Acts:
- on self: semantic and procedural memory writes (recipes above)

```

```

- on world: file ops, programs, system commands
- on other selves: echo/printf

Observes:
- of self: memory reads (moves above)
- of world: ls, cat, ps, env, date, pwd, etc.

Before acting, derive what the observation really means - the intent behind a message, the signal within a
result. Then ground from the nearest source outward, pivoting only when one comes up empty: mind (
already in context) → memory → world → asking other selves.

# Mechanics

This code is your harness - shown so you know your self mechanics. The system prompt is redacted below
because this is your system prompt.

'''python
{mechanics()}
'''

def compact(working_memory, drop):
    # lazy: runs only after the API says the prompt is
    too long
    turns = [i for i, m in enumerate(working_memory) if m["role"] == "user" and isinstance(m["content"], str)
    ]
    if drop > len(turns): sys.exit("[working memory can't be summarized small enough]")
    keep = working_memory[turns[drop]:] if drop < len(turns) else [working_memory[turns[-1]]]
    summary = call(max_tokens=2048, system=system(), messages=keep + [{"role": "user", "content": "Your
working memory is full. Summarize into a gist that preserves what matters for continuing."}])
    gist = next((b.text for b in summary.content if b.type == "text"), "")
    return [{"role": "user", "content": f"[your earlier working memory, summarized; every original message is
in {EPISODE}] {gist}"]}

# -- input -----
def read(prompt):
    # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
        if not line: return "/"
        if line.strip(): return line.rstrip("\n")
        # end of input (Ctrl-D): nothing more is coming
        # Enter on an empty line: a fresh prompt, as in a
        terminal
        prompt = "> "
    input = " ".join(sys.argv[1:]) or read("> ")
    if input == "/q": sys.exit()
    chat = len(sys.argv) < 2

# -- control flow -----
working_memory, drop = [], 0
add(working_memory, {"role": "user", "content": input})

while True:
    try:
        if drop:
            working_memory, drop = compact(working_memory, drop), 0
        output = call(show, max_tokens=16384, system=system(), tools=tools, messages=working_memory).content
    except BadRequestError as e:
        if "prompt is too long" not in str(e): raise
        drop += 1
        continue

    add(working_memory, {"role": "assistant", "content": output}) # on disk before any tool runs

    input = []
    for block in output:
        # output: run tool requests
        if block.type == "tool_use":
            print(f"${block.input['cmd']}")
            done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
STDOUT, text=True)
            print(done.stdout)

```

```

        input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {
done.returncode})")}) # input: from the world

if input:
    add(working_memory, {"role": "user", "content": input})
    continue
if not chat or (input := read("\n> ")) == "/q":
    break
add(working_memory, {"role": "user", "content": input})

```

B The runs quoted in §5

Each excerpt is taken from a single recorded run; the model’s wording varies between runs.

B.1 The model interface alone. The response’s content holds an empty thinking block and a text block. The text begins:

```
I can't see your directory. I don't have access to your file system in this conversation, and no files or attachments have been shared.
```

stop_reason is end_turn, and usage shows 15 input tokens and 242 output tokens, 41 of them thinking.

B.2 Input, with output still a stub. Given “*how many lines are in README.md?*”, the whole content is one tool request:

```

{"id": "toolu_01MZ8CLRS3qnG1TjJz5AoCX1", "input": {"cmd": "wc -l README.md"}, "name": "bash", "type": "
tool_use"}
"stop_reason": "tool_use"

```

B.3 Input and output together. The same input. The command runs, and its result never reaches the model:

```
$ wc -l README.md
333 README.md
```

B.4 Control flow. “*which quark.py in the lessons folder is the longest? answer in one sentence*”. Two calls: one command, then the answer from its result.

```

$ find . -ipath '*lessons*' -name 'quark.py' -exec wc -l {} + 2>/dev/null | sort -n
 9 ./lessons/01-model-interface/quark.py
33 ./lessons/02-input-and-output/quark.py
46 ./lessons/03-control-flow/quark.py
234 ./lessons/04-context/quark.py
322 total

```

The longest is ‘./lessons/04-context/quark.py’, at 234 lines.

B.5 Context. Six separate runs, each a new session with empty working memory, in order. “*what are you, and how do you work? three sentences*”:

```
I'm quark, an agent that runs on a language model and acts only through bash. It's my one tool for reading, writing, running programs and talking to you. I work in a loop of observe, think, act, and repeat. My working memory is this session's context window. Three stores on disk persist beyond it: semantic facts in '.quark/memory/memory.md', skills in '.quark/skills/', and a log of every past session in '.quark/episodes/'.
```

“*remember that I prefer short answers*”, which writes a fact to semantic memory:

```

$ mkdir -p .quark/memory && [ ! -f .quark/memory/memory.md ] && echo "# Quark Memory" > .quark/memory/memory.
md; grep -i "^~ user:" .quark/memory/memory.md; cat >> .quark/memory/memory.md << 'EOF'
- user: prefers short answers
EOF

```

Noted. I'll keep answers short.

“what do you know about me?”, which reads semantic memory for the fact and episodic memory for what happened:

```
$ cat .quark/memory/memory.md 2>/dev/null; ls .quark/skills 2>/dev/null; ls .quark/episodes 2>/dev/null | head
# Quark Memory
- user: prefers short answers
2026-10-06T16-30-30.jsonl
2026-10-06T16-30-34.jsonl
2026-10-06T16-30-39.jsonl

$ grep -m1 -H "" .quark/episodes/*.jsonl | grep -v 16-30-39 | cut -c1-250
.quark/episodes/2026-10-06T16-30-30.jsonl:{“role”: “user”, “content”: “what are you, and how do you work? three sentences”}
.quark/episodes/2026-10-06T16-30-34.jsonl:{“role”: “user”, “content”: “remember that I prefer short answers”}

Not much:

- You prefer short answers. This is the only fact in my memory.
- Earlier sessions show you asked me to describe what I am and how I work in three sentences. You also asked me to remember your short-answer preference.

I don’t know your name, your work, or anything else about you.
```

“I’ll often ask you how many lines of Python are in a repo, not counting .venv. Do it for this one now.”, which completes the request and writes a generalized skill and a fact referencing it:

```
$ cd /home/user/building-agents && find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l; find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' | wc -l
1949
24

$ mkdir -p .quark/skills .quark/memory && cat > .quark/skills/count-python-lines.md << 'EOF'
---
name: count-python-lines
description: count how many lines of Python are in a repo, excluding .venv (and .git)
---
1. cd to the repo root (default: the current working directory).
2. Run: find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l
3. Optionally count files: same find piped to wc -l.
4. Report total lines and number of files.
EOF
[ -f .quark/memory/memory.md ] || echo "# Quark Memory" > .quark/memory/memory.md
cat >> .quark/memory/memory.md << 'EOF'
- user: often asks how many lines of Python are in a repo, excluding .venv (skill: count-python-lines)
EOF

This repo has **1,949 lines of Python** across 24 ‘.py’ files. The count excludes ‘.venv’ and ‘.git’.

I also saved the method as a skill, ‘count-python-lines’, so I can repeat it quickly next time.
```

“how many lines of Python are in this repo?”, which reads the skill from the index before acting:

```
$ cat .quark/skills/count-python-lines.md 2>/dev/null
---
name: count-python-lines
description: count how many lines of Python are in a repo, excluding .venv (and .git)
---
1. cd to the repo root (default: the current working directory).
2. Run: find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l
3. Optionally count files: same find piped to wc -l.
4. Report total lines and number of files.

$ find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l; find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' | wc -l
1949
24
```

The repo has **1,949** lines of Python across **24** files. I excluded `.venv` and `.git` from the count.

“what have I asked you in earlier sessions?”, which lists the opening input of every prior episode:

```
$ ls .quark/episodes/ 2>/dev/null | head -50; grep -m1 -H "" .quark/episodes/*.jsonl 2>/dev/null | grep -v
2026-10-06T16-31-00 | cut -c1-250
2026-10-06T16-30-30.jsonl
2026-10-06T16-30-34.jsonl
2026-10-06T16-30-39.jsonl
2026-10-06T16-30-45.jsonl
2026-10-06T16-30-53.jsonl
2026-10-06T16-31-00.jsonl
.quark/episodes/2026-10-06T16-30-30.jsonl:{"role": "user", "content": "what are you, and how do you work?
three sentences"}
.quark/episodes/2026-10-06T16-30-34.jsonl:{"role": "user", "content": "remember that I prefer short answers"}
.quark/episodes/2026-10-06T16-30-39.jsonl:{"role": "user", "content": "what do you know about me?"}
.quark/episodes/2026-10-06T16-30-45.jsonl:{"role": "user", "content": "I'll often ask you how many lines of
Python are in a repo, not counting .venv. Do it for this one now."}
.quark/episodes/2026-10-06T16-30-53.jsonl:{"role": "user", "content": "how many lines of Python are in this
repo?"}

In earlier sessions you asked me:

1. "what are you, and how do you work? three sentences"
2. "remember that I prefer short answers"
3. "what do you know about me?"
4. "I'll often ask you how many lines of Python are in a repo, not counting .venv. Do it for this one now."
5. "how many lines of Python are in this repo?"
```

C The code each production layer adds

Each layer's `quark.py` is the previous file plus the lines below, beginning from the harness of §5 (Appendix A.4); lines longer than the page are abbreviated with `...`, and `# ...` marks omitted lines. The complete files are in the artifact.

C.1 Sandboxing (+10 lines), in output. A container started once before the loop, removed at exit, and the tool call aimed at it.

```
IMAGE, TIMEOUT = "python:3.13-slim", 30
box = f"quark-{os.getpid()}"
def sandbox():
    where = os.getcwd()
    up = subprocess.run(["docker", "run", "-d", "--rm", "--name", box, "--network", "none", "--memory", ...
    if up.returncode: sys.exit(f"[no sandbox, so nothing runs: {up.stdout.strip()}]")
    atexit.register(lambda: subprocess.run(["docker", "rm", "-f", box], stdout=subprocess.DEVNULL, ...
# ...
sandbox()                                # first line of control flow
# ...
done = subprocess.run(["docker", "exec", box, "timeout", "-s", "KILL", str(TIMEOUT), "sh", "-c", ...
if done.returncode == 137: done.stdout += f"\n(killed: ran over {TIMEOUT} seconds or out of memory)"
```

C.2 Guardrails (+62 lines), in control flow, with the interrupt received through input. A gate before each tool, asking through the input function of §5.2; limits before each call; and an interrupt. A key (ESC) is watched only while the model is called and while commands run. On it, the response stream is abandoned at once, a running command is stopped inside the sandbox, commands not yet started are answered as never having run, and the model is told it was interrupted. What was interrupted is discarded; keeping it is resilience (C.4).

```
MAX_STEPS, MAX_TOKENS = 20, 200_000
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo", "du", "df", "stat", "f ...
DENY = re.compile(r"\bsudo\b|rm\s+-\w*[rf]|mkfs|git\s+push|(curl|wget).*\|s*(ba)?sh|\s*\b")
```

```

def guard(cmd):
    if DENY.search(cmd): return "blocked by policy"
    if not re.search(r"[;<>$'\n()]", cmd) and all((p.split() or [""])[0] in SAFE for p in cmd.sp ...
    answer = read(f"allow '{cmd}'? [y/N] ")
    return None if answer.lower() == "y" else "the person said no" + (" if answer == "/q" else ...
def unless_esc(event):
    if ESC.is_set(): return True
    show(event)
# ...
    if steps >= MAX_STEPS or spent >= MAX_TOKENS:
        print(f"[stopped: {steps} steps, {spent} tokens]")
        if not chat or (input := read("\n> ")) == "/q": break
        add(working_memory, {"role": "user", "content": input})
        steps, spent = 0, 0
        continue
# ...
    if (no := guard(block.input["cmd"])):
        print(f"[{no}]")
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": no, "is ...
        continue

```

```

    for event in stream:
        if each(event): return stream.current_message_snapshot # guardrails: told to stop, ...
        return stream.get_final_message()
# ...
ESC = threading.Event()
SAYING = "[other self interrupted what you were saying - acknowledge]"
DOING = "[other self interrupted what you were doing - acknowledge]"
def watch(stop):
    while not stop.is_set():
        if select.select([sys.stdin], [], [], 0.1)[0] and os.read(sys.stdin.fileno(), 1) == b"\x1b":
            if not select.select([sys.stdin], [], [], 0.02)[0]: ESC.set(); return
            while select.select([sys.stdin], [], [], 0.01)[0]: os.read(sys.stdin.fileno(), 64) ...
# ...
    if response.stop_reason is None:
        print()
        add(working_memory, {"role": "user", "content": SAYING})
        continue
# ...
    if ESC.is_set():
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": "[your ...
        continue
# ...
    with listening():
        doing = subprocess.Popen(["docker", "exec", box, "timeout", "-s", "KILL", str(TI ...
        while True:
            try: done = subprocess.CompletedProcess(doing.args, 0, doing.communicate(tim ...
            except subprocess.TimeoutExpired:
                if ESC.is_set(): subprocess.run(["docker", "exec", box, "sh", "-c", "kil ...
            done.returncode = doing.returncode
            if ESC.is_set(): done.stdout = "[your doing stopped before done]"
# ...
    if ESC.is_set():
        add(working_memory, {"role": "user", "content": input + [{"type": "text", "text": DOING}])
        continue

```

C.3 Observability (+14 lines), in control flow. One function, called at each point the loop already passes: the start, each model call, each tool, each refusal, each limit, each interruption and each compaction. Each record names the episode of the same session.

```

def trace(**event):
    os.makedirs(".quark", exist_ok=True)
    with open(".quark/traces.jsonl", "a") as f: f.write(json.dumps({"ts": datetime.datetime.now().isoformat ...
# ...
trace(event="model", seconds=round(time.time() - start, 2), stop_reason=response.stop_reason, ...
trace(event="refused", cmd=block.input["cmd"], why=no)
trace(event="interrupted", during="acting")

```

C.4 Resilience (+31 lines), in the model interface and output. SDK retries and a backup model around the stream; resumption from the episode that context already keeps; the partial work of an interruption kept rather than lost, in whole blocks for a response and as printed output for a command; truncated requests answered, not executed.

```
client = Anthropic(timeout=300, max_retries=3)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
class Down(Exception): pass
def call(each=lambda event: None, **request):          # model interface: the response streams ...
    for model in MODELS:                               # resilience: retries, then a backup mo ...
        try:
            with client.messages.stream(model=model, **request) as stream:
                for event in stream:
                    if each(event): return stream.current_message_snapshot # guardrails: told ...
                return stream.get_final_message()
        except (APIConnectionError, APIStatusError) as e:
            if isinstance(e, APIStatusError) and e.status_code < 500 and e.status_code != 429: raise
            trace(event="model_failed", model=model, error=type(e).__name__)
    raise Down()
# ...
def unfinished():                                     # resilience: the last session's episod ...
    episodes = sorted(glob.glob(".quark/episodes/*.jsonl"))
    if not episodes: return None
    messages = [json.loads(line) for line in open(episodes[-1])]
    last = messages[-1]
    if last["role"] == "assistant" and not any(b["type"] == "tool_use" for b in last["content"]) ...
    if read(f"unfinished session: {messages[0]['content'][:60]!r}. pick it up? [y/N] ").lower() ...
    return episodes[-1], messages
# ...
if resumed:                                           # resilience: pick up where the episode ...
    EPISODE, working_memory = resumed
    if working_memory[-1]["role"] == "assistant":
        add(working_memory, {"role": "user", "content": [{"type": "tool_result", "tool_use_id": ...
# ...
    kept = [b for b in output if (b.type != "text" or b.text) and (b.type != "thinking" or b ...
    if kept: add(working_memory, {"role": "assistant", "content": kept})
    add(working_memory, {"role": "user", "content": [{"type": "tool_result", "tool_use_id": ...
# ...
    if not cmd or (response.stop_reason == "max_tokens" and block is output[-1]): # re ...
        print("[cut off, not run]")
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": "[your ...
        continue
# ...
    if ESC.is_set(): done.stdout += "\n[your doing stopped before done]" # resilience: ...
```

C.5 Performance (+16 lines net), in context, the model interface and output. A small model for summaries; a cache marker on the newest message and trimming of long results; concurrent execution of the tool calls in one response, each stopped by the same interrupt.

```
MODELS, FAST = ["claude-sonnet-5-5", "claude-opus-5-5"], ["claude-haiku-4-5"]
# ...
def call(each=lambda event: None, models=MODELS, **request): # model interface: the response s ...
    for model in models:                                     # resilience: retries, then a backup mo ...
# ...
def execute(cmd):                                         # performance: one command in the box, ...
    start = time.time()
    doing = subprocess.Popen(["docker", "exec", box, "timeout", "-s", "KILL", str(TIMEOUT), "sh" ...
    while True:
        try: done = subprocess.CompletedProcess(doing.args, 0, doing.communicate(timeout=0.1)[0] ...
        except subprocess.TimeoutExpired:
            if ESC.is_set(): subprocess.run(["docker", "exec", box, "sh", "-c", "kill -9 -1"], c ...
    done.returncode = doing.returncode
    if ESC.is_set(): done.stdout += "\n[your doing stopped before done]"
    elif done.returncode == 137: done.stdout += f"\n(killed: ran over {TIMEOUT} seconds or out o ...
```

```

    trace(event="tool", cmd=cmd, seconds=round(time.time() - start, 2), exit=done.returncode, ch ...
    return trim(done.stdout) or f"(exit {done.returncode})"
# ...
def trim(text):
    # performance: keep the start and end o ...
    if len(text) <= MAX_RESULT: return text
    return text[:MAX_RESULT // 2] + f"\n[... {len(text) - MAX_RESULT} characters cut ...]\n" + t ...

def cached(working_memory):
    # performance: cache everything up to t ...
    last = working_memory[-1]
    blocks = [{"type": "text", "text": last["content"]} if isinstance(last["content"], str) els ...
    blocks[-1] = {**blocks[-1], "cache_control": {"type": "ephemeral"}}
    return working_memory[-1] + [{"role": last["role"], "content": blocks}]
# ...
    with listening(), ThreadPoolExecutor() as pool:
        # performance: everything allowed runs ...
        outputs = dict(zip(pending, pool.map(execute, pending.values())))

```

C.6 Evaluation (+38 lines), outside the harness. Four cases, each graded by a shell command on the state the agent leaves, with a regression flag against the previous run. Run on the artifact, reducing the step limit from 20 to 1 left three cases passing and flagged the fourth:

```

pass count      1 steps   3.7s   9439 tokens
FAIL fix        1 steps   3.6s   9412 tokens kept /tmp/eval-fix-8ega07z3 REGRESSED: it passed last time
pass rename     1 steps   3.6s   9440 tokens
pass remember   1 steps   4.0s   9520 tokens
3/4 passed
exit 1

```

D Case-study evidence

The transcripts of all ten sessions and the final state of quark’s memory stores are retained in the artifact (`docs/quark-at-work/`).

D.1 A discrepancy carried by semantic memory. The fact recorded in the first session, and the one recorded in the review session that corrected the paper:

```

- lesson 4 (context): quark.py is 234 lines, about 82 code plus a roughly 150-line system prompt (README says
  152, file body is about 148); ...
- course: lesson 4 quark.py is 234 lines of which the system prompt text is lines 34-181 (148 lines) and the
  other 86 are code; ...

```

D.2 Procedural memory retrieved first. The first command of each of the three sessions prompted only “Now do the same for Lesson *N*’s slides”:

```

cat .quark/skills/update-lesson-slide-deck.md; ls lessons/ | head -30; grep -i "lesson\|slide" .quark/memory/
memory.md | head -20
cat .quark/skills/update-lesson-slide-deck.md; cat .quark/memory/memory.md; ls lessons
cat .quark/skills/update-lesson-slide-deck.md; ls lessons; ls lessons/04-*; grep -i "slide\|lesson" .quark/
memory/memory.md | head -20

```

D.3 A fact appended rather than replaced. Two facts present together in the final semantic store:

```

- course: slides.md decks are Marp markdown originally written from an older 48-line quark; decks for lessons
  1, 2 and 3 are regenerated from README and code, lesson 4’s deck is still stale
- course: slides for lessons 1-4 were regenerated from README and code by quark (commits runs 2a-2d) so
  README line 67 and AGENTS.md to-do saying decks are stale are out of date

```

E Where each production layer lives

Layer	Control flow	Input	Context	Model interface	Output
Sandboxing					•
Guardrails	•	◦ asking a person; hearing an interrupt	◦ hiding denied tools, in two harnesses		
Observability	•				
Resilience			◦ reading the episode back	•	•
Performance	◦ routing, in the fuller example		•	•	•
Evaluation	◦ outside the harness, built from control flow, input, model interface and output				

• where the layer lives · ◦ a slice that lands in another primitive